BeaconKit 아키텍처 분석 - 10. 노드 API 및 REST 서버

1. 노드 API 개요

BeaconKit의 노드 API는 **이더리움 비콘 노드 API 표준**을 구현하여 외부 클라이언트가 노드 상태를 조회하고 상호작용할 수 있게 합니다. 이는 검증자 클라이언트, 모니터링 도구, 탐색기 등이 노드와 통신하는 주요 인터페이스입니다.

1.1 API 서버 구조

```
┌─────────────────────────────────────────────────────────────┐
| API Server |
| (HTTP REST 서버) |
├─────────────────┬─────────────────┬──────────────────────────┤
| Middleware | Handlers | Backend |
| (요청/응답 처리) | (API 엔드포인트)| (데이터 접근 계층) |
├─────────────────┼─────────────────┼──────────────────────────┤
| • CORS | • Beacon API | • StorageBackend |
| • Logging | • Config API | • StateProcessor |
| • Error Handling| • Debug API | • ConsensusService |
| • Rate Limiting | • Node API | • Genesis State Cache |
└─────────────────┴─────────────────┴──────────────────────────┘
```

1.2 지원 API 카테고리

Beacon API: 비콘 체인 상태 및 블록 정보

Config API: 체인 설정 및 사양 정보

Debug API: 디버깅 및 내부 상태 정보

Events API: 실시간 이벤트 스트리밍

Node API: 노드 상태 및 네트워크 정보

Proof API: 머클 증명 및 검증

Builder API: 블록 빌더 관련 기능

Validator API: 검증자 관련 기능

2. Server 구조 및 초기화

2.1 Server 구조체

```go
type Server struct {
config Config // 서버 설정
logger log.Logger // 로거
middleware *middleware.Middleware // 미들웨어 스택

b *backend.Backend // 데이터 접근 백엔드
// 테스트용으로 노출된 getter
beaconHandler *beaconapi.Handler // 비콘 API 핸들러
}
```

2.2 서버 초기화

```go
func  New(
config Config,
logger log.Logger,
// 핸들러 백엔드 구축을 위한 속성들
storageBackend *storage.Backend,
sp *core.StateProcessor,
cs chain.Spec,
cmtCfg *cmtcfg.Config,
// consensusService는 API가 노드 상태에 접근하고
// 히스토리컬 쿼리를 포함한 모든 종류의 쿼리를 수행할 수 있게 함
consensusService types.ConsensusService,
) *Server {
apiLogger := logger
if !config.Logging {
apiLogger = noop.NewLogger[log.Logger]()
}

mware := middleware.NewDefaultMiddleware(apiLogger)

// 핸들러들을 인스턴스화하고 미들웨어에 라우트 등록
b := backend.New(storageBackend, sp, cs, cmtCfg, consensusService)
```

```go
beaconHandler := beaconapi.NewHandler(b, cs, apiLogger)

// 각 API 카테고리별 핸들러 등록

mware.RegisterRoutes(beaconHandler.RouteSet())

mware.RegisterRoutes(builderapi.NewHandler(apiLogger).RouteSet())

mware.RegisterRoutes(configapi.NewHandler(cs, apiLogger).RouteSet())

mware.RegisterRoutes(debugapi.NewHandler(b, apiLogger).RouteSet())

mware.RegisterRoutes(eventsapi.NewHandler(apiLogger).RouteSet())

mware.RegisterRoutes(nodeapi.NewHandler(b, apiLogger).RouteSet())

mware.RegisterRoutes(proofapi.NewHandler(b, apiLogger).RouteSet())


return &Server{

config: config,

logger: logger,

middleware: mware,

b: b,

beaconHandler: beaconHandler,

}

}
```

2.3 서버 시작 및 중지

```go
func (s *Server) Start(ctx context.Context) error {

if !s.config.Enabled {
```

```go
return nil
}

// 모든 관련 node-api 백엔드 데이터를 사전 로드 및 캐시
if err := s.b.LoadData(ctx); err != nil {
return fmt.Errorf("failed loading api backend data: %w", err)
}

go s.start(ctx)
return nil
}

func (s *Server) start(ctx context.Context) {
errCh := make(chan error)
go func() {
errCh <- s.middleware.Run(s.config.Address)
}()
for {
select {
case err := <-errCh:
s.logger.Error(err.Error())
case <-ctx.Done():
return
```

```
}

}

}


func (s *Server) Stop() error {

return  s.b.Close()

}
```

3. Backend - 데이터 접근 계층

3.1 Backend 구조

```
type Backend struct {

sb *storage.Backend // 저장소 백엔드

cs chain.Spec // 체인 사양

cmtCfg *cmtcfg.Config // CometBFT 설정 (제네시스 데이터 가져오기용)

node types.ConsensusService // 합의 서비스


// 제네시스 관련 데이터

sp GenesisStateProcessor // 제네시스 상태 재생성용

db dbm.DB // 제네시스 상태 저장용 인메모리 DB

muSt sync.RWMutex // 제네시스 상태 안전한 초기화/복사용 뮤텍스

cms storetypes.CommitMultiStore
```

```
genesisState *state.StateDB // API 요청 서비스용 제네시스 데이터 캐시
}
```

3.2 데이터 로딩 프로세스

```
func (b *Backend) LoadData(_ context.Context) error {
switch err := b.node.IsAppReady(); {
case err == nil:
// 체인이 준비됨, 제네시스 로딩 시작
return b.loadGenesisState()
case errors.Is(err, cometbft.ErrAppNotReady):
// 어쨌든 시작, 나중에 제네시스 상태 초기화
return nil
default:
return fmt.Errorf("unable to check whether app is ready: %w", err)
}
}
```

3.3 제네시스 상태 로딩

```
func (b *Backend) loadGenesisState() error {
b.muSt.Lock()
defer b.muSt.Unlock()
```

```go
if b.genesisState != nil {
// 제네시스 상태가 이미 초기화됨
return nil
}

// STEP 1: 제네시스 바이트 로드
genesisData, err := b.parseGenesisBytes()
if err != nil {
return err
}

// STEP 2: 제네시스 저장소 생성
if err = b.initGenesisState(); err != nil {
return fmt.Errorf("backend data loading:%w", err)
}

// STEP 3: 상태 프로세서를 통해 제네시스 재처리
// 자체 상태에서 수행되므로 안전하며, 상태 프로세서는
// 제네시스 처리 중 EVM 호출을 하지 않음
if _, err = b.sp.InitializeBeaconStateFromEth1(
b.genesisState,
genesisData.GetDeposits(),
```

```
genesisData.GetExecutionPayloadHeader(),
genesisData.GetForkVersion(),
);  err != nil {
return  fmt.Errorf("failed processing genesis: %w", err)
}


_ = b.cms.Commit() // 인메모리 DB에는 실제로 필요하지 않음
return  nil
}
```

3.4 제네시스 데이터 파싱

```
func (b *Backend) parseGenesisBytes() (ctypes.Genesis, error) {
// STEP 1: 파일에서 앱 제네시스 로드
appGenesis, err := genutiltypes.AppGenesisFromFile(b.cmtCfg.GenesisFile())
if  err != nil {
return ctypes.Genesis{}, fmt.Errorf("failed loading app genesis from file: %w",
}
// STEP 2: 제네시스 문서로 변환
gen, err := appGenesis.ToGenesisDoc()
if  err != nil {
return ctypes.Genesis{}, fmt.Errorf("failed parsing app genesis: %w", err)
}
```

```go
// STEP 3: 앱 상태 언마샬링
var genesisState map[string]json.RawMessage
if err = json.Unmarshal(gen.AppState, &genesisState); err != nil {
return ctypes.Genesis{}, fmt.Errorf("failed to unmarshal genesis state: %w", err
}
// STEP 4: 비콘 제네시스 데이터 추출
data := []byte(genesisState["beacon"])
genesisData := ctypes.Genesis{}
if err = json.Unmarshal(data, &genesisData); err != nil {
return ctypes.Genesis{}, fmt.Errorf("failed to unmarshal genesis data: %w", err)
}
return genesisData, nil
}
```

4. API 핸들러 구조

4.1 핸들러 인터페이스

```go
type Handler interface {
RouteSet() RouteSet // 라우트 집합 반환
}

type RouteSet []Route
```

```go
type Route struct {
Method  string  // HTTP 메서드
Path  string  // URL 경로
Handler echo.HandlerFunc // 핸들러 함수
}
```

4.2 Beacon API 핸들러 예시

```go
type Handler struct {
backend Backend
cs chain.Spec
logger log.Logger
}

func  NewHandler(backend Backend, cs chain.Spec, logger log.Logger) *Handler {
return &Handler{
backend: backend,
cs: cs,
logger: logger,
}
}
```

```go
func (h *Handler) RouteSet() RouteSet {
return RouteSet{
{
Method: "GET",
Path: "/eth/v1/beacon/genesis",
Handler: h.GetGenesis,
},
{
Method: "GET",
Path: "/eth/v1/beacon/states/:state_id/root",
Handler: h.GetStateRoot,
},
{
Method: "GET",
Path: "/eth/v1/beacon/blocks/:block_id",
Handler: h.GetBlock,
},
// ... 더 많은 엔드포인트들
}
}
```

4.3 핸들러 구현 예시

```go
func (h *Handler) GetGenesis(c echo.Context) error {
// 제네시스 데이터 조회
genesis, err := h.backend.GetGenesis(c.Request().Context())
if err != nil {
return handleError(c, err)
}
// JSON 응답 반환
return c.JSON(http.StatusOK, map[string]interface{}{
"data": genesis,
})
}

func (h *Handler) GetStateRoot(c echo.Context) error {
stateID := c.Param("state_id")
// 상태 루트 조회
root, err := h.backend.GetStateRoot(c.Request().Context(), stateID)
if err != nil {
return handleError(c, err)
}
return c.JSON(http.StatusOK, map[string]interface{}{
"execution_optimistic": false,
"finalized": true,
```

```
"data": map[string]interface{}{
"root": root,
},
})
}
```

5. 미들웨어 시스템

5.1 미들웨어 구조

```
type Middleware struct {
echo *echo.Echo
logger log.Logger
}

func  NewDefaultMiddleware(logger log.Logger) *Middleware {
e := echo.New()
// 기본 미들웨어 설정
e.Use(middleware.Logger())
e.Use(middleware.Recover())
e.Use(middleware.CORS())
// 커스텀 미들웨어
e.Use(requestLoggingMiddleware(logger))
```

```
e.Use(errorHandlingMiddleware(logger))
return &Middleware{
echo: e,
logger: logger,
}
}
```

5.2 라우트 등록

```
func (m *Middleware) RegisterRoutes(routes RouteSet) {
for _, route := range routes {
m.echo.Add(route.Method, route.Path, route.Handler)
}
}

func (m *Middleware) Run(address string) error {
return m.echo.Start(address)
}
```

5.3 커스텀 미들웨어

요청 로깅 미들웨어:

```
func requestLoggingMiddleware(logger log.Logger) echo.MiddlewareFunc {
```

```go
return func(next echo.HandlerFunc) echo.HandlerFunc {
return func(c echo.Context) error {
start := time.Now()
err := next(c)
logger.Info("API request",
"method", c.Request().Method,
"path", c.Request().URL.Path,
"status", c.Response().Status,
"duration", time.Since(start),
)
return err
}
}
}
```

오류 처리 미들웨어:

```go
func errorHandlingMiddleware(logger log.Logger) echo.MiddlewareFunc {
return func(next echo.HandlerFunc) echo.HandlerFunc {
return func(c echo.Context) error {
err := next(c)
if err != nil {
logger.Error("API error",
"path", c.Request().URL.Path,
```

```
"error", err,
)
// 표준 오류 응답 반환
return handleAPIError(c, err)
}
return nil
}
}
}
```

6. 상태 조회 및 컨텍스트 관리

6.1 상태 식별자 처리

API는 다양한 상태 식별자를 지원합니다:

- `head` : 현재 헤드 상태
- `genesis` : 제네시스 상태
- `finalized` : 최종화된 상태
- `justified` : 정당화된 상태
- `<slot>` : 특정 슬롯의 상태
- `<root>` : 특정 상태 루트

```
func (b *Backend) GetStateByID(ctx context.Context, stateID string) (*state.Sta
switch stateID {
```

```
case "head":
return b.sb.StateFromContext(ctx), nil
case "genesis":
return b.getGenesisState(), nil
case "finalized":
return b.getFinalizedState(ctx), nil
case "justified":
return b.getJustifiedState(ctx), nil
default:
// 슬롯 번호나 루트로 파싱 시도
if slot, err := parseSlot(stateID); err == nil {
return b.getStateBySlot(ctx, slot), nil
}
if root, err := parseRoot(stateID); err == nil {
return b.getStateByRoot(ctx, root), nil
}
return nil, ErrInvalidStateID
}
}
```

6.2 컨텍스트 기반 상태 접근

```go
func (b *Backend) StateFromContext(ctx context.Context) *state.StateDB {
// 컨텍스트에서 포크 정보 추출
forkVersion := extractForkVersionFromContext(ctx)
// 해당 포크에 맞는 상태 반환
return  b.sb.StateFromContext(ctx)
}
```

7. 이벤트 스트리밍

7.1 Server-Sent Events (SSE)

```go
type EventHandler struct {
logger log.Logger
events  chan Event
}

func (h *EventHandler) StreamEvents(c echo.Context) error {
// SSE 헤더 설정
c.Response().Header().Set("Content-Type", "text/event-stream")
c.Response().Header().Set("Cache-Control", "no-cache")
c.Response().Header().Set("Connection", "keep-alive")
// 이벤트 스트리밍
for {
```

```
select {
case event := <-h.events:
if err := writeSSEEvent(c.Response(), event); err != nil {
return err
}
case <-c.Request().Context().Done():
return nil
}
}
}
```

7.2 지원 이벤트 타입

- `head` : 새로운 헤드 블록
- `block` : 새로운 블록
- `attestation` : 새로운 증명
- `voluntary_exit` : 자발적 탈퇴
- `finalized_checkpoint` : 최종화된 체크포인트
- `chain_reorg` : 체인 재구성

8. 오류 처리 및 응답 형식

8.1 표준 오류 응답

```go
type ErrorResponse struct {
Code int `json:"code"`
Message string `json:"message"`
Data any `json:"data,omitempty"`
}


func handleAPIError(c echo.Context, err error) error {
var (
code int
message string
)
switch {
case errors.Is(err, ErrNotFound):
code = http.StatusNotFound
message = "Resource not found"
case errors.Is(err, ErrInvalidRequest):
code = http.StatusBadRequest
message = "Invalid request"
case errors.Is(err, ErrInternalError):
code = http.StatusInternalServerError
message = "Internal server error"
default:
```

```
code = http.StatusInternalServerError

message = "Unknown error"

}

return  c.JSON(code, ErrorResponse{

Code: code,

Message: message,

})

}
```

8.2 성공 응답 형식

```
type SuccessResponse struct {

Data any `json:"data"`

ExecutionOptimistic  bool  `json:"execution_optimistic,omitempty"`

Finalized  bool  `json:"finalized,omitempty"`

}
```

9. 성능 최적화

9.1 데이터 캐싱

제네시스 상태 캐싱:

```
// 제네시스 상태를 메모리에 캐시하여 반복 조회 최적화

func (b *Backend) getGenesisState() *state.StateDB {
```

```
b.muSt.RLock()
defer b.muSt.RUnlock()
return b.genesisState
}
```

응답 캐싱:

```
// 자주 요청되는 데이터에 대한 응답 캐싱
type ResponseCache struct {
cache *lru.Cache[string, CachedResponse]
ttl time.Duration
}
```

9.2 비동기 처리

백그라운드 데이터 로딩:

```
func (s *Server) Start(ctx context.Context) error {
// 백그라운드에서 데이터 사전 로드
go func() {
if err := s.b.LoadData(ctx); err != nil {
s.logger.Error("Failed to load backend data", "error", err)
}
}()
return s.startServer(ctx)
```

```
}
```

10. 보안 및 인증

10.1 CORS 설정

```
func configureCORS() echo.MiddlewareFunc {
return middleware.CORSWithConfig(middleware.CORSConfig{
AllowOrigins: []string{"*"},
AllowMethods: []string{echo.GET, echo.POST, echo.PUT, echo.DELETE},
AllowHeaders: []string{echo.HeaderOrigin, echo.HeaderContentType, echo.HeaderAcc
})
}
```

10.2 Rate Limiting

```
func rateLimitingMiddleware() echo.MiddlewareFunc {
return middleware.RateLimiter(middleware.NewRateLimiterMemoryStore(
rate.Limit(100), // 초당 100 요청
))
}
```

11. 모니터링 및 메트릭

11.1 API 메트릭

```
type APIMetrics struct {
requestCount prometheus.Counter
requestDuration prometheus.Histogram
errorCount prometheus.Counter
activeConnections prometheus.Gauge
}
```

11.2 헬스 체크

```
func (h *Handler) HealthCheck(c echo.Context) error {
status := map[string]interface{}{
"status": "healthy",
"timestamp": time.Now().Unix(),
"version": version.Version,
}
return  c.JSON(http.StatusOK, status)
}
```

12. 설계 원칙

12.1 표준 준수

- 이더리움 비콘 노드 API 표준 구현
- RESTful API 설계 원칙 준수
- OpenAPI 스펙 호환성

12.2 성능 최적화

- 데이터 캐싱을 통한 응답 시간 단축
- 비동기 처리를 통한 처리량 향상
- 효율적인 상태 조회 메커니즘

12.3 안정성

- 포괄적인 오류 처리
- 우아한 성능 저하
- 리소스 누수 방지

12.4 확장성

- 모듈형 핸들러 구조
- 플러그인 가능한 미들웨어
- 수평적 확장 지원

이 노드 API 시스템은 BeaconKit이 외부 세계와 상호작용하는 주요 인터페이스로, 검증자, 모니터링 도구, 탐색기 등이 노드 상태에 접근할 수 있게 해주는 핵심 컴포넌트입니다.